Master of Science in Industrial and Applied Mathematics, 2026/2027

Machine Learning for Physics I

Physics-informed neural networks (PINNs)

Filippo Masi*

Laboratoire Jean Kuntzmann (LJK), TRIPOP team
Université Grenoble Alpes, Inria, MIAI | Artificial Intelligence & Mechanics (AIM)







*filippo.masi@inria.fr

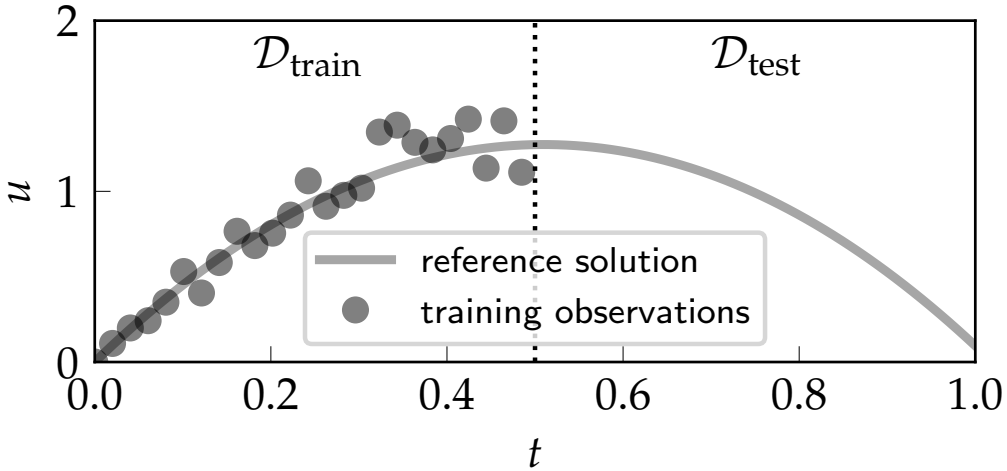
Outline

1. Why PINNs?
2. Partial differential equations
3. Physics-informed neural networks: forward problems
4. Physics-informed neural networks: inverse problems
5. Why and when PINNs fail
6. Hard constraints (a short *tour*)

Why PINNs?

Re-discovering ballistics

Problem: identify the evolution of the trajectory (1D) of a projectile from observations.

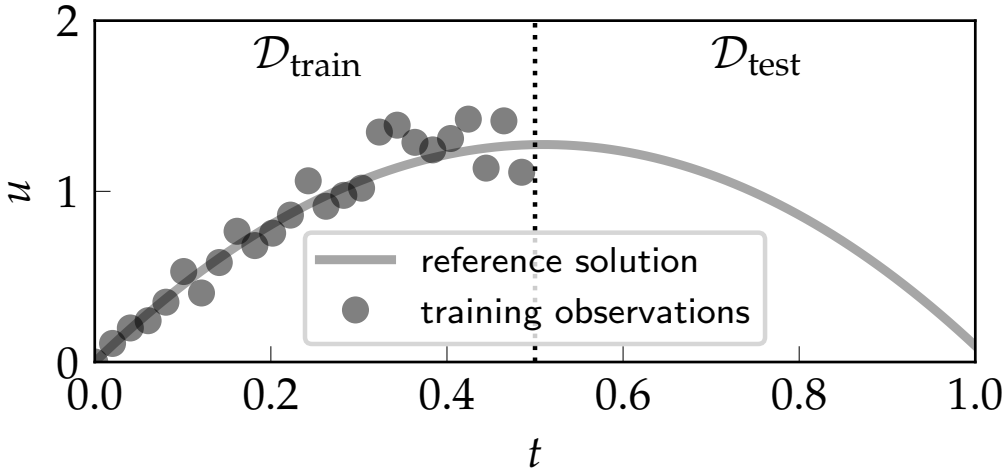


- we observe discrete $u^{(i)} \in \mathcal{D}_{\text{train}}$
- we know the corresponding ODE is

$$u_{tt}(t) + g = 0, \quad g = 9.81 \text{ m/s}^2,$$
$$u(0) = u_0,$$
$$\dot{u}(0) = v_0.$$

Re-discovering ballistics

Problem: identify the evolution of the trajectory (1D) of a projectile from observations.

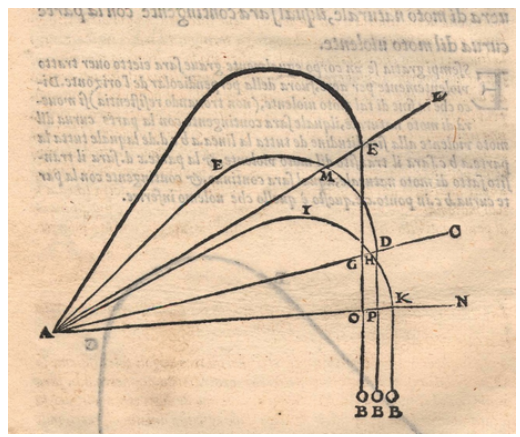


- we observe discrete $u^{(i)} \in \mathcal{D}_{\text{train}}$

Nicolò Tartaglia, *La nova scientia* (1558)

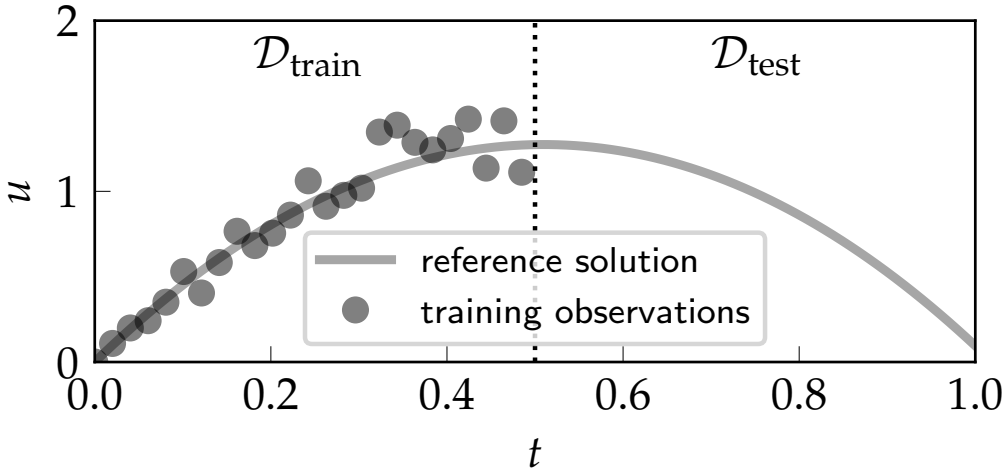






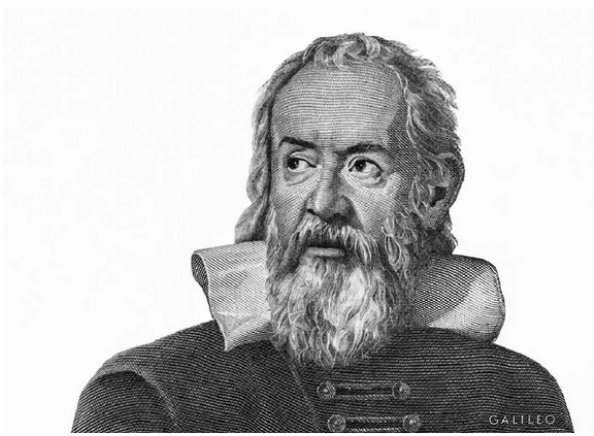
Re-discovering ballistics

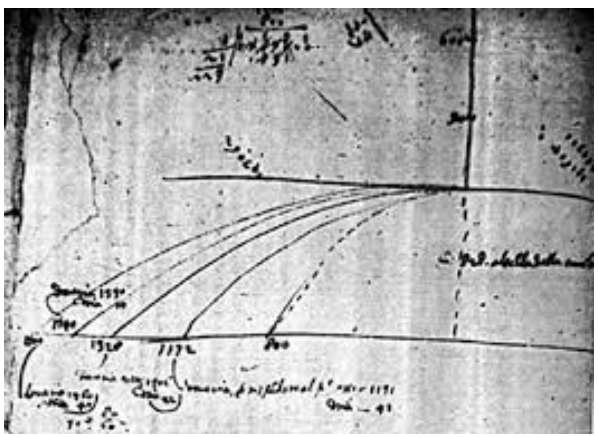
Problem: identify the evolution of the trajectory (1D) of a projectile from observations.



- we observe discrete $u^{(i)} \in \mathcal{D}_{\text{train}}$

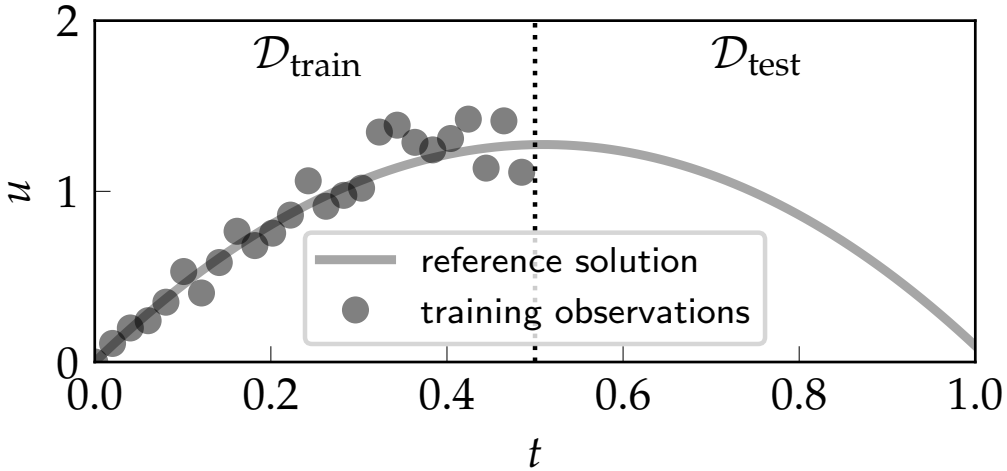
Galileo Galilei, *De motu antiquiora* (1589)





Re-discovering ballistics

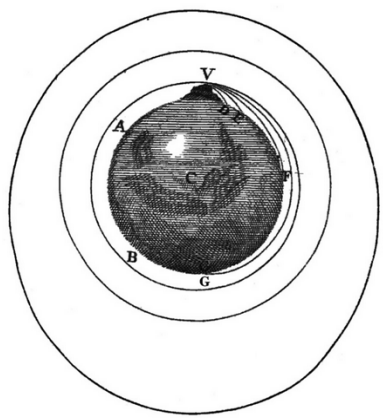
Problem: identify the evolution of the trajectory (1D) of a projectile from observations.



- we observe discrete $u^{(i)} \in \mathcal{D}_{\text{train}}$

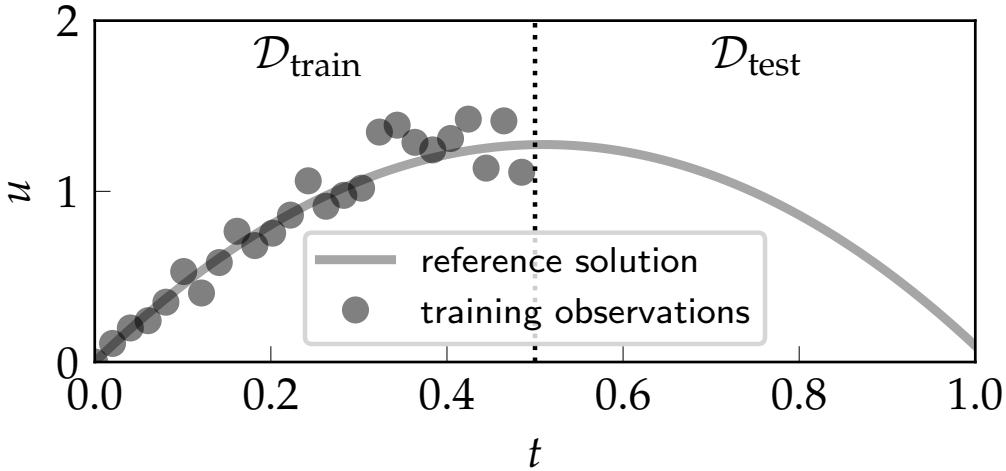
Isaac Newton, *On the system of the world* (1731)





Re-discovering ballistics

Problem: identify the evolution of the trajectory (1D) of a projectile from observations.



- we observe discrete $u^{(i)} \in \mathcal{D}_{\text{train}}$

Hands-on

Goals:

1. train a multilayer-perceptron NN such that $u^{(i)} \approx u_\theta(t^{(i)})$, $0 \leq t \leq 0.5$ s,
2. train its physics-informed version (*Galilean!*, quadratic trajectory), i.e., $u_{\theta,tt} + \gamma = 0$,
3. evaluate the predictions for distributionally shifted inputs, i.e., $0 \leq t \leq 1$ s,
4. assess the performances with a rectified linear unit activation (ReLU).

Data generation and sampling

```python
import numpy as np
import torch
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

dtype = torch.float32
torch.manual_seed(123)
np.random.seed(123)
# Physical parameters used only to generate the synthetic reference solution.
u0 = 0.0        # m
v0 = 5.0        # m/s
g = 9.81        # m/s^2
t_start = 0.0; t_end = 1.0

def projectile_motion(t, state):
    """First-order system equivalent to u_tt + g = 0."""
    u, v = state
    return [v, -g]

X = np.linspace(t_start, t_end, 100, dtype=np.float32)
sol = solve_ivp(projectile_motion, [t_start, t_end], [u0, v0], t_eval=X)
y = sol.y[0]

# Subsample the first half of the trajectory.
t_i_np = X[0:50:2].copy()
u_i_np = y[0:50:2].copy()
sigma = 0.10
u_i_np += sigma * np.random.normal(0.0, 1.0, len(u_i_np)) * u_i_np
u_i_np[0] = 0.0
t_i = torch.tensor(t_i_np, dtype=dtype).view(-1, 1) # inputs
u_i = torch.tensor(u_i_np, dtype=dtype).view(-1, 1) # outputs
```

Building a MLP neural network

```python
class NeuralNet(torch.nn.Module):
    """Fully-connected neural network for u_theta(t)."""

    def __init__(
        self,
        hidden_layers=(12, 12), dtype=torch.float32, learn_gamma=False,
    ):
        super().__init__()

        layers = []
        in_dim = 1

        for width in hidden_layers:
            layers.append(torch.nn.Linear(in_dim, width, dtype=dtype))
            layers.append(torch.nn.Tanh())
            in_dim = width

        layers.append(torch.nn.Linear(in_dim, 1, dtype=dtype))

        self.NN = torch.nn.Sequential(*layers)

        # Unknown constant acceleration used only by the PINN.
        self.gamma = torch.nn.Parameter(
            torch.tensor(1.0, dtype=dtype),
            requires_grad=learn_gamma,
        )

    def forward(self, t):
        return self.NN(t)


model_nn = NeuralNet(dtype=dtype)
```

Training utils

```
def data_loss(model):
    """Mean-squared error on the observed trajectory."""
    u_pred = model(t_i)
    return torch.mean((u_pred - u_i) ** 2)

def make_data_closure(model, optimizer):
    """Closure used by either Adam or L-BFGS."""
    def closure():
        optimizer.zero_grad(set_to_none=True)
        loss = data_loss(model)
        loss.backward()
        return loss

    return closure
```

Training the MLP network

– Training

```
# --- Adam phase ------------------------------------------------------------
n_adam = 40_000
report_every = 1_000

optimizer = torch.optim.Adam([p for p in model_nn.parameters() if p.requires_grad], lr=1e-2)
closure = make_data_closure(model_nn, optimizer)

for step in range(n_adam):
    loss = optimizer.step(closure)

# --- L-BFGS refinement -----------------------------------------------------
optimizer = torch.optim.LBFGS([p for p in model_nn.parameters() if p.requires_grad],
                              lr=1.0, max_iter=50, line_search_fn="strong_wolfe")
closure = make_data_closure(model_nn, optimizer)
optimizer.step(closure)
```

Training the MLP network

– Training

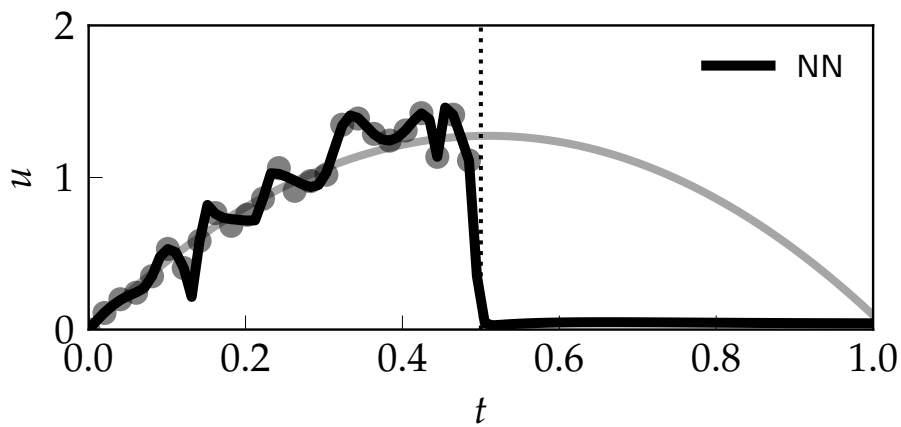
```
# --- Adam phase ------------------------------------------------------------
n_adam = 40_000
report_every = 1_000

optimizer = torch.optim.Adam([p for p in model_nn.parameters() if p.requires_grad], lr=1e-2)
closure = make_data_closure(model_nn, optimizer)

for step in range(n_adam):
    loss = optimizer.step(closure)

# --- L-BFGS refinement -----------------------------------------------------
optimizer = torch.optim.LBFGS([p for p in model_nn.parameters() if p.requires_grad],
                              lr=1.0, max_iter=50, line_search_fn="strong_wolfe")
closure = make_data_closure(model_nn, optimizer)
optimizer.step(closure)
```

– Inference



Physics-informed neural network

The *physics-informed neural net* version uses two sources of information

- empirical loss on data: $L_D = \frac{1}{N} \sum_i \left(u_\theta(t^{(i)}) - u^{(i)} \right)^2$
- learning bias: $L_r = \frac{1}{N_c} \sum_k r_\theta^2(t_c^{(k)})$, $r_\theta(t_c^{(k)}) = u_{\theta,tt}(t^{(k)}) + \gamma$, $t^{(k)}$ collocation points.

```
# Collocation points cover the entire interval on which physics is enforced.

t_k = torch.linspace(0.0, 1.0, 20, dtype=dtype).view(-1, 1).requires_grad_(True)
lambda_PDE = 0.5

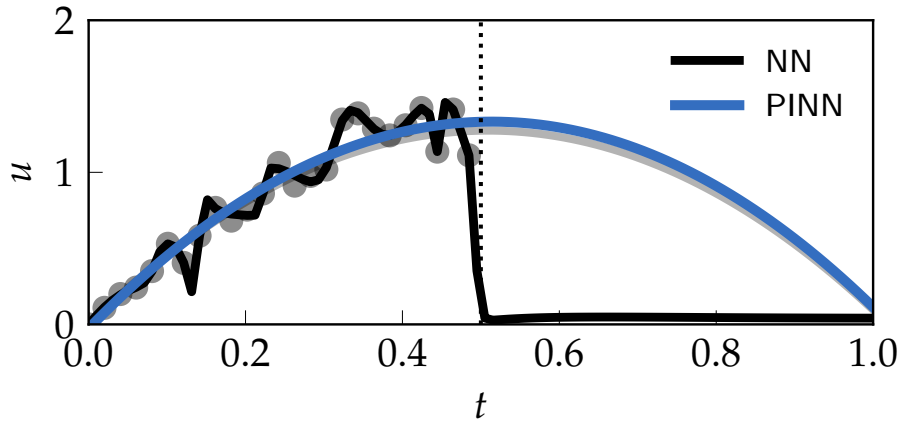
def physics_loss(model):
    """
    Compute the mean squared residual of u_tt + gamma = 0 at the collocation points.
    """
    u_k = model(t_k)
    du_dt = torch.autograd.grad(u_k, t_k, grad_outputs=torch.ones_like(u_k), create_graph=True)[0]
    d2u_dt2 = torch.autograd.grad(du_dt, t_k, grad_outputs=torch.ones_like(du_dt), create_graph=True,)[0]
    residual = d2u_dt2 + model.gamma
    return torch.mean(residual**2)

def pinn_loss(model):
    return data_loss(model) + lambda_PDE * physics_loss(model)

def make_pinn_closure(model, optimizer):
    def closure():
        optimizer.zero_grad(set_to_none=True)
        loss = pinn_loss(model)
        loss.backward()
        return loss
    return closure
```

Physics-informed neural network versus neural network

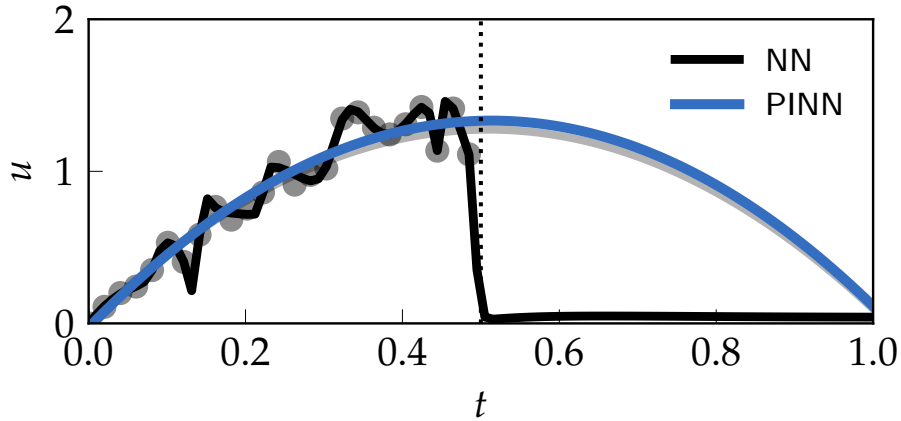
– Inference: NN versus PINN (notebook run: $\gamma \approx 9.62$ m/s^2)



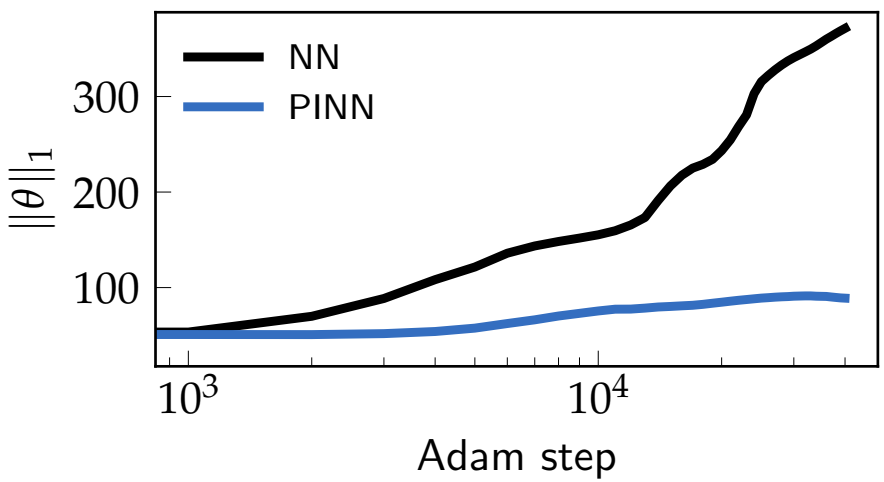
Long story short: *If Galileo had used PINN, he would have discovered the equations of motion for ballistics 140 years before Isaac did.*

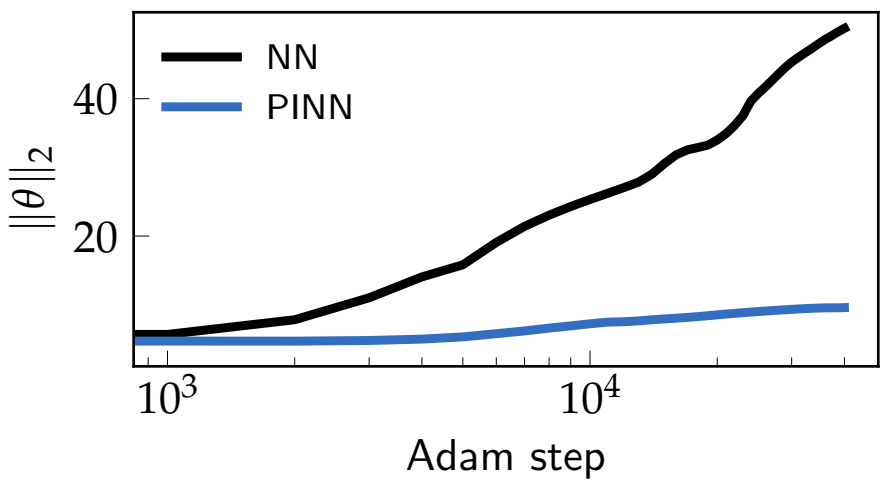
Physics-informed neural network versus neural network

– Inference: NN versus PINN (notebook run: $\gamma \approx 9.62$ m/s^2)



Long story short: *If Galileo had used PINN, he would have discovered the equations of motion for ballistics 140 years before Isaac did.*

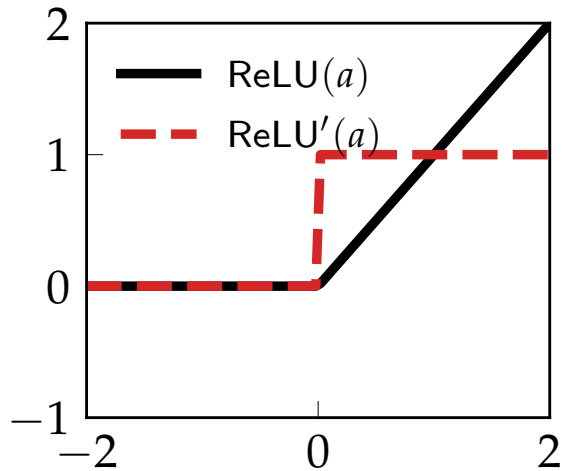




On activation functions in PINNs

A standard MLP with affine layers and ReLU activations is piecewise affine in t:

$$u_{\theta,tt}(t) = 0 \quad \text{almost everywhere.}$$



For generic collocation points, AD therefore gives $r_\theta(t;\gamma) = \gamma$, $L_r = \gamma^2$.

Known gravity

- $L_r = g^2$ and the second-derivative residual cannot teach the required curvature.

Unknown gravity

- The physics loss favours $\gamma \to 0$ and a good data fit can coexist with a wrong inferred acceleration.

Regularity of the ansatz functions

Use smooth activations such as tanh for classical second-order residuals.

The question after ballistics

We inferred one trajectory from observed positions and a law of motion. What if no new trajectory observations are available?

Forward problem

Given an equation, coefficients, initial state, and boundary conditions, determine the state field u.

Inverse problem

Given incomplete noisy observations, infer an unknown coefficient or state while respecting the equation.

A PINN represents a candidate field with a neural network and tests its physical admissibility through differential residuals. This lecture explains when that reasoning works, when it fails, and what can be enforced exactly.

The generalization

The same unknown can be a time curve $u(t)$, a vectorial spatial field $\boldsymbol{u}(\boldsymbol{x})$, or a space–time field $u(\boldsymbol{x}, t)$. A PINN changes the trial field and differential operator accordingly.

Partial differential equations

Partial differential equation (PDE) models

- Complex physical laws are founded on simple principles, including symmetry, conservation of mass, and conservation of energy, which are expressed as PDEs.
- Let $\Omega \subset \mathbb{R}^d$ be an open physical domain, with boundary $\partial\Omega$. Let $u : \Omega \to \mathbb{R}$ be a scalar physical quantity. A PDE of order k has the form

$$F\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), D^2u(\mathbf{x}), \ldots, D^k u(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

  where F is the equation, λ denotes its parameters, and f is the source term.
- Let $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_d) \in \mathbb{N}_0^d$, the set of all partial derivatives of u of total order $\boldsymbol{\alpha}$ is

$$D^{\boldsymbol{\alpha}} u(\mathbf{x}) = \left\{ \frac{\partial^{|\boldsymbol{\alpha}|} u}{\partial x_1^{\alpha_1} \cdots \partial x_d^{\alpha_d}} \;\middle|\; |\boldsymbol{\alpha}| = \alpha_1 + \cdots + \alpha_d \right\}.$$

- A derivative with respect to $x_{i_1}, \ldots, x_{i_n}$ is denoted as $\frac{\partial^n u}{\partial x_{i_1} \cdots \partial x_{i_n}} = u_{x_{i_1} \cdots x_{i_n}}$.
- First- and second-order derivatives of u are organized into the gradient vector and Hessian matrix:

$$Du = \begin{bmatrix} u_{x_1} \\ \vdots \\ u_{x_d} \end{bmatrix}, \qquad D^2 u = \begin{bmatrix} u_{x_1 x_1} & \cdots & u_{x_1 x_d} \\ \vdots & \ddots & \vdots \\ u_{x_d x_1} & \cdots & u_{x_d x_d} \end{bmatrix}.$$

- If u has continuous second partial derivatives, then $u_{x_i x_j} = u_{x_j x_i}$ and $D^2 u = (D^2 u)^{\mathsf{T}}$ (cf. Clairaut's theorem).

Partial differential equation (PDE) models

There are several different classifications of PDEs. A PDE of order k is

- linear, if F is linear in all its arguments, with coefficients that may depend on $\boldsymbol{x}$.
- semi-linear, if F is linear in the elements of $D^k u$, with coefficients that may depend on $\boldsymbol{x}$.
- quasi-linear, if F is linear in the elements of $D^k u$, with coefficients that may depend on $\boldsymbol{x}$ and all partial derivatives of order less than k.
- nonlinear, if F is not linear (which includes semi-linear and quasi-linear PDEs).

Another classification describes the principal (highest-derivative) part of a second-order PDE:

- Hyperbolic, $u_{tt} - c^2 \Delta u = f$ (wave equation).
  The solution of a nonlinear hyperbolic PDE can develop discontinuities (shocks) starting from smooth initial conditions, at which time the function is no longer differentiable and thus stops being a classic solution.
- Parabolic, $u_t - \alpha \Delta u = f$ (heat equation).
  Parabolic PDE tend to diffuse and smooth many kind of initial conditions.
- Elliptic, $-\Delta u = f$ (Poisson equation).
  The solution of an elliptic PDE is static and thus is not affected by such considerations.

Partial differential equation (PDE) models

- The case $d = 1$

$$F\left(u, x, \frac{du}{dx}, \frac{d^2u}{dx^2}, \ldots, \frac{d^ku}{dx^k}; \lambda\right) = f(x), \qquad x \in \Omega \subset \mathbb{R}$$

  is an ordinary differential equation of order k.
- The extension to the vectorial case $\boldsymbol{u} = (u_1, \ldots, u_m)$ defines a PDE/ODE system. For example, a first-order PDE system reads

$$\mathbf{u} = (u^1, \ldots, u^m), \qquad F_i\big(\mathbf{u}(\mathbf{x}), \mathbf{x}, D\mathbf{u}(\mathbf{x}); \lambda_i\big) = f_i(\mathbf{x}), \quad i = 1, \ldots, p$$

- The PDE alone generally does not select a unique physical solution: additional information on the unknown u or its derivatives must be provided.
- It is usually the case that physical considerations provide this information on all or part of the boundary $\partial\Omega$ of the physical domain.
- These boundary conditions can be written as

$$G\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), D^2u(\mathbf{x}), \ldots, D^ku(\mathbf{x}); \lambda\big) = g(\mathbf{x}),$$

  which must be satisfied at all $\boldsymbol{x} \in \Gamma \subseteq \partial\Omega$, for a given function G and boundary data g.
- In a time evolution problem, data on the boundary $t = 0$ is called an initial condition.

Physics-informed neural networks: forward problems

Physics-informed neural networks: forward problems

One of the most basic techniques in physics-informed machine learning is constraining the output of a deep neural network by adding a term to the loss function that penalizes a lack of agreement with a PDE.

- In a forward problem, the model parameters λ and the defining data are known:

$$F\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^k u(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

$$G\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^m u(\mathbf{x})\big) = g(\mathbf{x}), \qquad \mathbf{x} \in \Gamma \subseteq \partial\Omega.$$

- This is therefore a pure PDE solving problem, in which the PINN is used as a traditional numerical method.
- The problem where there is incomplete information, which must be estimated from sample data, is referred to the inverse problem.

Physics-informed neural networks: forward problems

One of the most basic techniques in physics-informed machine learning is constraining the output of a deep neural network by adding a term to the loss function that penalizes a lack of agreement with a PDE.

- In a forward problem, the model parameters λ and the defining data are known:

$$F\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^k u(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

$$G\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^m u(\mathbf{x})\big) = g(\mathbf{x}), \qquad \mathbf{x} \in \Gamma \subseteq \partial\Omega.$$

- This is therefore a pure PDE solving problem, in which the PINN is used as a traditional numerical method.
- The problem where there is incomplete information, which must be estimated from sample data, is referred to the inverse problem.
- Let the unknown $u(\boldsymbol{x})$ be approximated by an ansatz $u_{\boldsymbol{\theta}}(\boldsymbol{x})$, consisting of a deep neural network with inputs $\boldsymbol{x} = (x_1, \ldots, x_d)$ and network weights $\boldsymbol{\theta}$.
- The ansatz $u_{\boldsymbol{\theta}}(\boldsymbol{x})$ is substituted into the PDE and BC and the task is to find the network weights $\boldsymbol{\theta}$ such that

$$F\big(u_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{x}, Du_{\boldsymbol{\theta}}(\mathbf{x}), \ldots, D^k u_{\boldsymbol{\theta}}(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega, \tag{1}$$

$$G\big(u_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{x}, Du_{\boldsymbol{\theta}}(\mathbf{x}), \ldots, D^m u_{\boldsymbol{\theta}}(\mathbf{x})\big) = g(\mathbf{x}), \qquad \mathbf{x} \in \Gamma \subseteq \partial\Omega. \tag{2}$$

  The values of the partial derivatives $D^\alpha u_{\boldsymbol{\theta}}(\boldsymbol{x})$ of the neural network output, for given values of $\boldsymbol{x}$ and $\boldsymbol{\theta}$, can be computed accurately and efficiently using AD.

Physics-informed neural networks: forward problems

- Consider the following loss functions

$$L_r(\boldsymbol{\theta}) = \int_\Omega \left| F\left(u_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{x}, Du_{\boldsymbol{\theta}}(\mathbf{x}), \ldots, D^k u_{\boldsymbol{\theta}}(\mathbf{x}); \lambda\right) - f(\mathbf{x}) \right|^2 d\boldsymbol{x}, \tag{3}$$

$$L_b(\boldsymbol{\theta}) = \int_\Gamma \left| G\left(u_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{x}, Du_{\boldsymbol{\theta}}(\mathbf{x}), \ldots, D^m u_{\boldsymbol{\theta}}(\mathbf{x})\right) - g(\mathbf{x}) \right|^2 d\boldsymbol{S}, \tag{4}$$

- We have $L_r(\boldsymbol{\theta})$, $L_b(\boldsymbol{\theta}) \geq 0$, and (1), (2) are satisfied: this implies that $u_{\boldsymbol{\theta}}(\boldsymbol{x})$ satisfies the PDE and the boundary conditions everywhere (outside a set of measure zero) if and only if $L_r(\boldsymbol{\theta}) = L_b(\boldsymbol{\theta}) = 0$ or, equivalently, $L_r(\boldsymbol{\theta}) + L_b(\boldsymbol{\theta}) = 0$.
- But, the integrals in (3) and (4) must be numerically approximated.
  The simplest way to do so, but not necessarily the only way, is to form Monte-Carlo estimates using a uniformly distributed random samples $\{\boldsymbol{x}_r^i\}_{i=i}^{N_r} \subset \Omega$ and $\{\boldsymbol{x}_b^i\}_{i=0}^{N_b} \subset \Gamma$

$$L_r(\boldsymbol{\theta}) \approx \frac{1}{N_r} \sum_{i=i}^{N_r} \left| F\left(u_{\boldsymbol{\theta}}(\mathbf{x}_r^i), \mathbf{x}_r^i, Du_{\boldsymbol{\theta}}(\mathbf{x}_r^i), \ldots, D^k u_{\boldsymbol{\theta}}(\mathbf{x}_r^i); \lambda\right) - f(\mathbf{x}_r^i) \right|^2$$

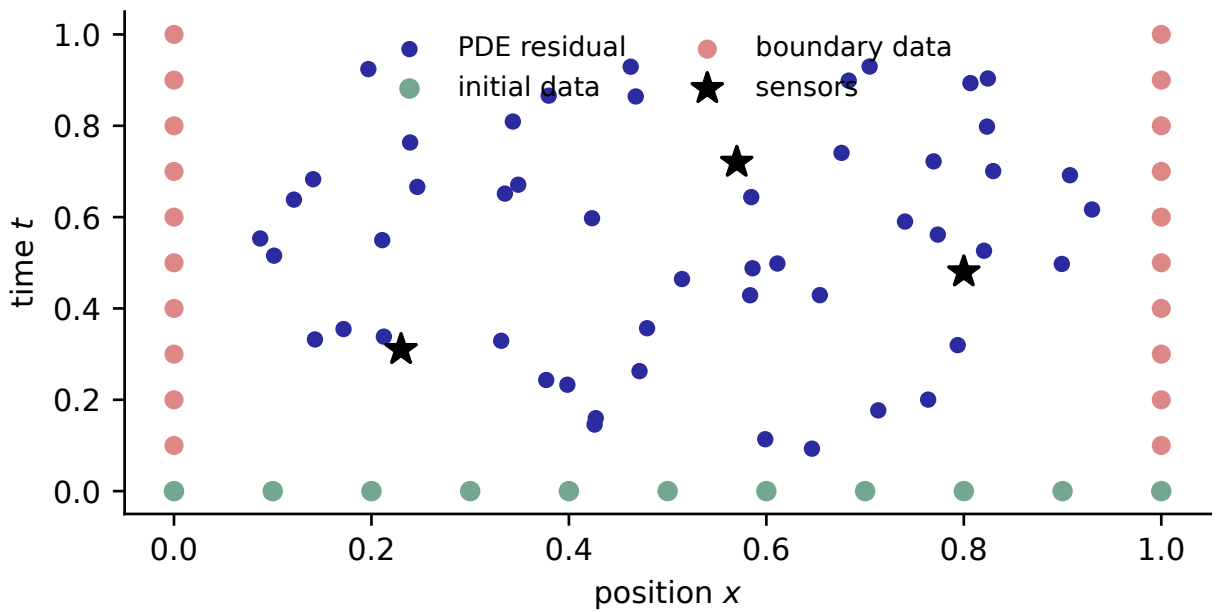
$$L_b(\boldsymbol{\theta}) \approx \frac{1}{N_b} \sum_{i=1}^{N_b} \left| G\left(u_{\boldsymbol{\theta}}(\mathbf{x}_b^i), \mathbf{x}_b^i, Du_{\boldsymbol{\theta}}(\mathbf{x}_b^i), \ldots, D^m u_{\boldsymbol{\theta}}(\mathbf{x}_b^i)\right) - g(\mathbf{x}_b^i) \right|^2$$

Physics-informed neural networks: forward problems

- The neural network is trained by minimizing the combined loss

$$L(\boldsymbol{\theta}) = L_r(\boldsymbol{\theta}) + L_b(\theta)$$

  using any of the optimization methods we have seen.
- PINN is a meshless method, as the collocation points $\{\boldsymbol{x}_r^i\}_{i=0}^{N_r}$ and boundary points $\{\boldsymbol{x}_b^i\}_{i=0}^{N_b}$ do not have to be located on regular meshes.
- Collocation points need not form a regular mesh and may be resampled during training. This forward problem requires no labeled solution values.
- The sample means estimate the normalized integrals for uniform independent sampling, and samples may be changed during training.



PINN training is a multi-objective problem

The issue is that PINN training is a multi-objective optimization problem, the solution of which is not unique $\rightarrow$ weighted compromise between satisfying the equation in the interior and satisfying the boundary data:

$$\min_{\theta}\big(L_r(\theta), L_b(\theta)\big) \quad \longrightarrow \quad \min_{\theta} L(\theta), \quad L(\boldsymbol{\theta}) = w_r L_r(\boldsymbol{\theta}) + w_b L_b(\boldsymbol{\theta}) \quad w_r, w_b > 0.$$

- The weighted sum is a *scalarization*: its weights select a training trade-off. Equal weights ($w_p = 1$) are a baseline, but equal numerical weights do not imply equal gradient influence and errors.
- These weights can be fixed or change during training, via a variety of methods, e.g.
  (1) Ad-hoc fixed weights (e.g., setting one of the weights to a large value to prioritize that component).
  (2) Change weights during training using gradient information ("learning rate annealing", Wang and Perdikaris, 2020).
  (3) Redistribute the weights according to the loss, putting more weight in the components that do not decrease (Liu and Wang, 2021).

These methods assign one weight to an entire loss component. Self-adaptive PINNs instead assign trainable weights to *individual coordinates*.

Self-adaptive PINNs: a weight at each point

Let u_θ be the network approximation. At interior collocation points $\mathbf{x}_i^r$ and boundary points $\mathbf{x}_j^b$, define

$$r_i(\boldsymbol{\theta}) \coloneqq \mathcal{F}[u_\theta](\mathbf{x}_i^r) - f(\mathbf{x}_i^r), \qquad b_j(\boldsymbol{\theta}) \coloneqq \mathcal{B}[u_\theta](\mathbf{x}_j^b) - g(\mathbf{x}_j^b),$$

where $\mathcal{F} = F(I, \boldsymbol{x}, D, D^2, \ldots, D^k; \lambda)$ and $\mathcal{B} = G(I, \boldsymbol{x}, D, D^2, \ldots, D^m)$.

Instead of assigning one weight to the entire PDE loss, assign a trainable weight to *each sampled point*:

$$\boldsymbol{w}_r = (w_{r,1}, \ldots, w_{r,N_r}), \qquad \boldsymbol{w}_b = (w_{b,1}, \ldots, w_{b,N_b}).$$

The weighted losses are

$$L_r(\boldsymbol{\theta}, \boldsymbol{w}_r) = \frac{1}{N_r} \sum_{i=1}^{N_r} m(w_{r,i}) \, |r_i(\boldsymbol{\theta})|^2, \qquad L_b(\boldsymbol{\theta}, \boldsymbol{w}_b) = \frac{1}{N_b} \sum_{j=1}^{N_b} m(w_{b,j}) \, |b_j(\boldsymbol{\theta})|^2.$$

Here $m : [0, \infty) \to [0, \infty)$ is a nonnegative *mask function*, for example $m(w) = w^2$. Initial-condition points can be weighted in the same way. The total loss is $L(\boldsymbol{\theta}, \boldsymbol{w}_r, \boldsymbol{w}_b) = L_r(\boldsymbol{\theta}, \boldsymbol{w}_r) + L_b(\boldsymbol{\theta}, \boldsymbol{w}_b)$.

- The network parameters $\boldsymbol{\theta}$ are updated to *decrease* the weighted equation and boundary residuals.
- The point weights $\boldsymbol{w}_r, \boldsymbol{w}_b$ are updated to *increase* the loss. A point that remains poorly fitted therefore receives more emphasis in subsequent network updates.

Gradient descent and ascent

At iteration k, use gradient descent for the network and gradient ascent for the point weights:

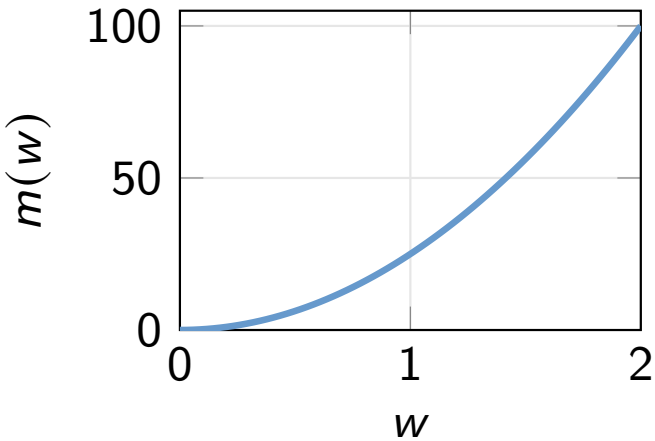
$$\begin{aligned}\boldsymbol{\theta}^{k+1} &= \boldsymbol{\theta}^k - \eta_k \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k, \boldsymbol{w}_r^k, \boldsymbol{w}_b^k),\\ \boldsymbol{w}_r^{k+1} &= \boldsymbol{w}_r^k + \rho_{r,k} \nabla_{\boldsymbol{w}_r} L(\boldsymbol{\theta}^k, \boldsymbol{w}_r^k, \boldsymbol{w}_b^k),\\ \boldsymbol{w}_b^{k+1} &= \boldsymbol{w}_b^k + \rho_{b,k} \nabla_{\boldsymbol{w}_b} L(\boldsymbol{\theta}^k, \boldsymbol{w}_r^k, \boldsymbol{w}_b^k),\end{aligned}$$

where $\eta_k, \rho_{r,k}, \rho_{b,k} > 0$.

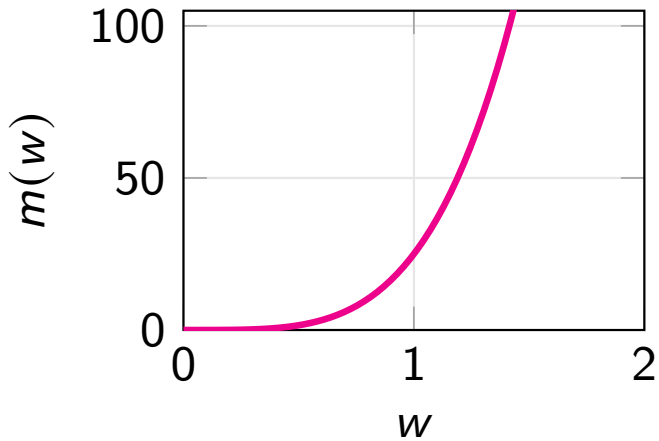
- A point whose residual stays large gains weight. The next network update places greater emphasis on reducing its error.
- With $m'(w) > 0$, weights are nondecreasing under gradient ascent; they stop changing at a point when its residual reaches zero. The ascent rates $\rho_{r,k}$ and $\rho_{b,k}$ determine how rapidly this emphasis changes.
- The method can help training focus on stubborn *sampled* points. It does not detect an unsampled region automatically: adaptive collocation may be still required.

Examples of self-adaptive mask functions

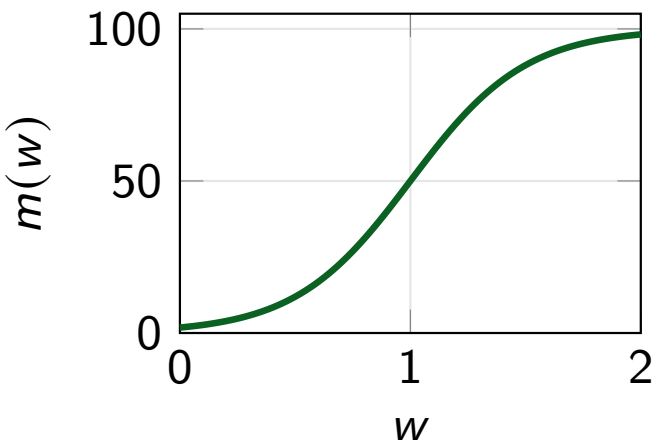
Polynomial: $25w^2$



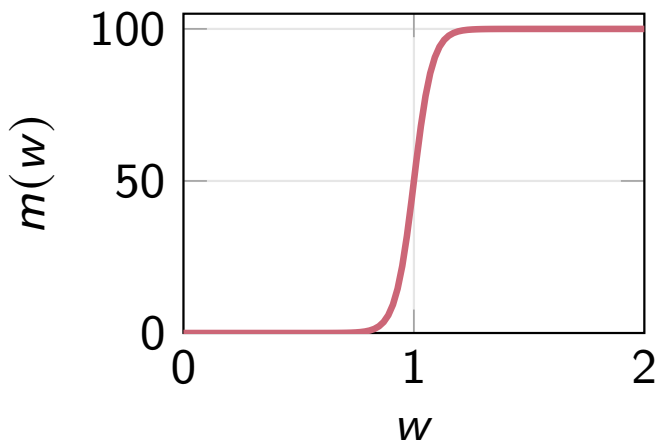
Polynomial: $25w^4$



Smooth logistic: $a = 4$



Sharp logistic: $a = 25$



Physics-informed neural networks: inverse problems

Forward and inverse problems

Consider a PDE with physical parameters $\boldsymbol{\lambda}$:

$$F\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^k u(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

$$G\big(u(\mathbf{x}), \mathbf{x}, Du(\mathbf{x}), \ldots, D^m u(\mathbf{x})\big) = g(\mathbf{x}), \qquad \mathbf{x} \in \Gamma \subseteq \partial\Omega.$$

- A *forward problem* consists in obtaining $u(\boldsymbol{x})$ from parameters $\boldsymbol{\lambda}$ and boundary conditions.
- An *inverse problem* consists of obtaining the value of unknown parameters from noisy data on $u(\boldsymbol{x})$.
  We observe the state imperfectly and seek the parameters that explain those observations.
- A well-posed forward problem has a solution that exists, is unique, and depends continuously on its defining data.
- An inverse problem usually lacks these properties: different parameters may explain the same observations, or small measurement errors may cause large changes in the estimate.

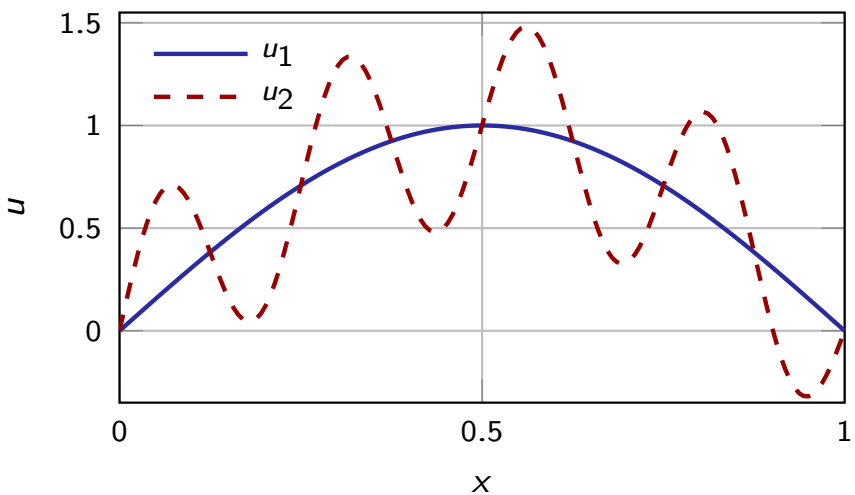
Why an inverse problem can be unstable: backward heat flow

Consider the heat equation with zero boundary values,

$$u_t(x,t) - \kappa u_{xx}(x,t) = 0, \qquad 0 < x < 1, \quad 0 < t < T, \qquad u(0,t) = u(1,t) = 0.$$

Suppose the final state $u(x,T)$ is measured and we wish to recover the initial state $u(x,0)$.

- This is an ill-posed problem, as there is an infinite number of initial states that lead to the same final state (this is similar to trying to find the initial shape of an ice cube after it has melted).
- In addition, the solution does not vary continuously with the data: it can be shown that the backward diffusion equation is equivalent to a forward diffusion equation with the final state as the initial condition, but with a negative diffusion coefficient, in which case any minute high-frequency noise in the initial condition (i.e., final state) immediately dominates the solution (cf. Neto, 2026).



Inverse PINNs

Consider the problem of obtaining the parameter λ in the PDE

$$F\big(u_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{x}, Du_{\boldsymbol{\theta}}(\mathbf{x}), \ldots, D^k u_{\boldsymbol{\theta}}(\mathbf{x}); \lambda\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

from data $\{\boldsymbol{x}_s^i, u^\epsilon(\boldsymbol{x}_s^i)\}_{i=0}^{N_s} \subseteq \Omega \times \mathbb{R}$ where $u^\epsilon(\boldsymbol{x}_s^i)$ are the noisy observations of $u(\boldsymbol{x}_s^i)$.

- Let $u_{\boldsymbol{\theta}}(\boldsymbol{x})$ be an ansatz deep network and let $\{\boldsymbol{x}_s^i\}_{i=0}^{N_s}$ be the set of collocation points.
- Consider the losses:

$$L_s(\boldsymbol{\theta}) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left| u_{\boldsymbol{\theta}}(\boldsymbol{x}_i^s) - u^\epsilon(\boldsymbol{x}_s^i) \right|^2$$

$$L_r(\boldsymbol{\theta}, \lambda) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| F\big(u_{\boldsymbol{\theta}}(\mathbf{x}_r^i), \mathbf{x}_r^i, Du_{\boldsymbol{\theta}}(\mathbf{x}_r^i), \ldots, D^k u_{\boldsymbol{\theta}}(\mathbf{x}_r^i); \lambda\big) - f(\mathbf{x}_r^i) \right|^2$$

- The neural network weights $\boldsymbol{\theta}$ and the parameter λ are simultaneously obtained by minimizing the combined loss

$$L(\boldsymbol{\theta}, \lambda) = L_s(\boldsymbol{\theta}) + L_r(\boldsymbol{\theta}, \lambda)$$

- when $\lambda : \Omega \to \mathbb{R}$ is a function, a parameter neural network $\lambda_{\boldsymbol{\theta}}(\boldsymbol{x})$ is required. Once $\lambda_{\boldsymbol{\theta}}$ is trained, it can be used to infer the estimated value of the parameter over any desired set of points in the domain.

Why and when PINNs fail

A small loss: which statement has actually been established?

$$r_\theta(\boldsymbol{x}) = F(u_\theta, \boldsymbol{x}, Du_\theta, \ldots; \lambda) - f(\boldsymbol{x}), \quad L(\boldsymbol{\theta}) = w_r L_r + w_b L_b + w_s L_s.$$

Again: $\boldsymbol{\theta}$ are network parameters, $\boldsymbol{w}$ are numerical weights, L_b includes initial/boundary conditions, and L_s is present when state observations are used.

1. *Representation:* does the trial family contain a sufficiently accurate solution, including the required derivatives? A good fit to u alone need not give good fluxes or residuals.
2. *Optimization:* does training find such a parameter vector? A representable solution may be inaccessible to the chosen initialization, objective scaling, or optimizer.
3. *Sampling:* is the residual small throughout the domain? Training evaluates

$$L_r = \frac{1}{|\Omega|} \int_\Omega |r_\theta|^2 \, d\boldsymbol{x}, \qquad L_r \approx \frac{1}{N_r} \sum_{i=1}^{N_r} |r_\theta(\boldsymbol{x}_r^i)|^2.$$

   The second quantity estimates the first: they are not identical!
4. *Stability:* do small equation and data errors imply a small error in the desired physical quantity? This depends on the mathematical problem and the relevant norms.

Finite collocation can completely miss a local error

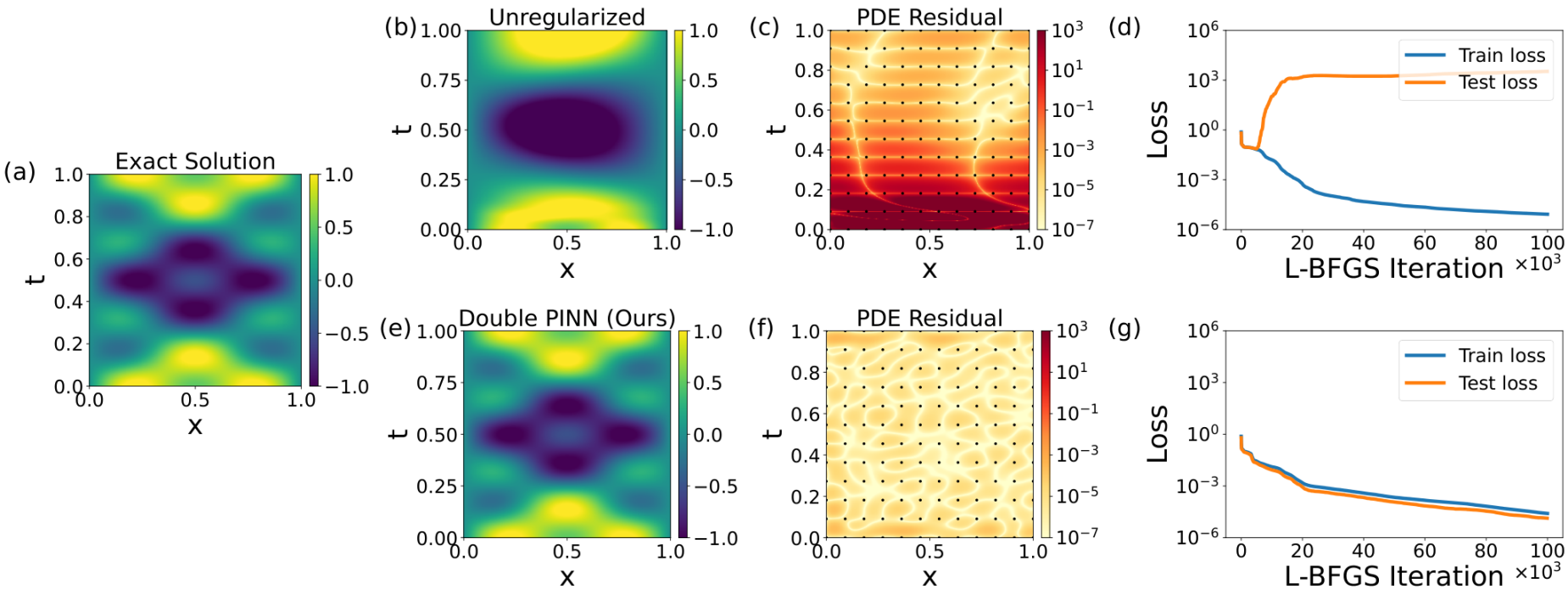
Wave eq: $u_{tt} - 4u_{xx} = 0,\ u(0,x) = \sin(\pi x) + \frac{1}{2}\sin(\beta\pi x),\ u_t(0,x) = 0,\ u(t,0) = u(t,1) = 0$

– Standard PINN:

$$L(\boldsymbol{\theta}) = L_r(\boldsymbol{\theta}) + \sum_{j=1}^{N_b} L_{b,j}(\boldsymbol{\theta}).$$

– Regularized PINN:

$$L(\boldsymbol{\theta}) = L_r(\boldsymbol{\theta}) + \frac{w_{\text{reg}}}{2}\left\|Dr_{\boldsymbol{\theta}}\right\|^2 + \sum_{j=1}^{N_b}\left[L_{b,j}(\boldsymbol{\theta}) + \frac{w_{\text{reg}}}{2}\left\|Db_{\boldsymbol{\theta},j}\right\|^2\right].$$



(Andersen, Matsubara, 2026)

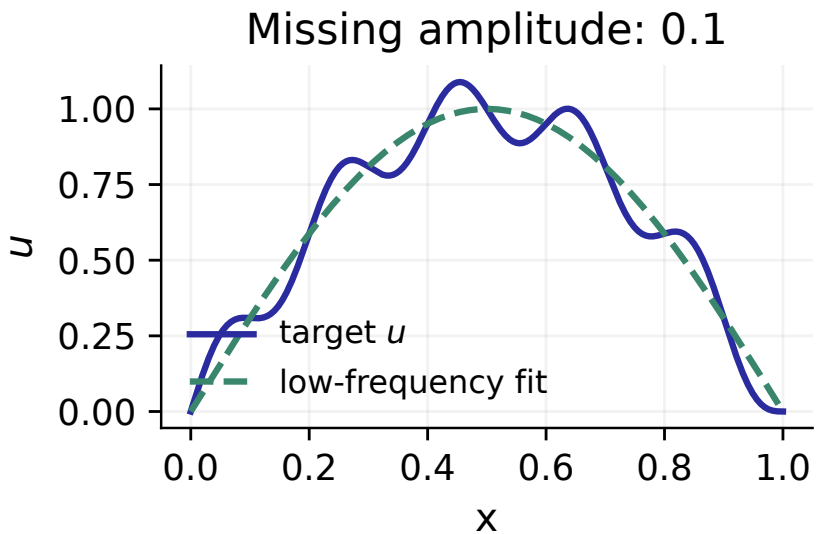
Consequence: lowering the loss on a fixed collocation set cannot, by itself, exclude a missed feature between its points.

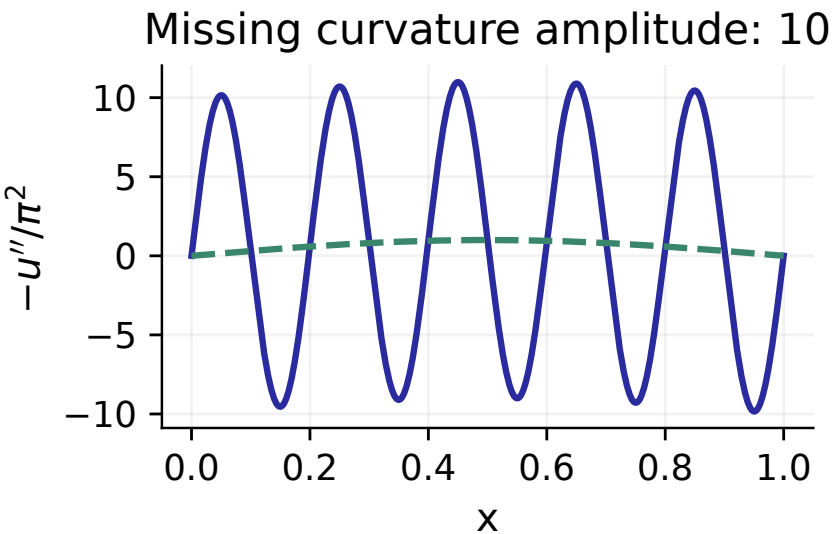
Spectral bias: a visually small error can be physically large

Many standard smooth networks preferentially fit low spatial frequencies during training. Consider

$$u(x) = \sin(\pi x) + 0.1\sin(10\pi x), \qquad -u''(x) = \pi^2 \sin(\pi x) + 10\pi^2 \sin(10\pi x).$$

The oscillatory component has only 10% of the field amplitude, but ten times the low-frequency curvature amplitude.

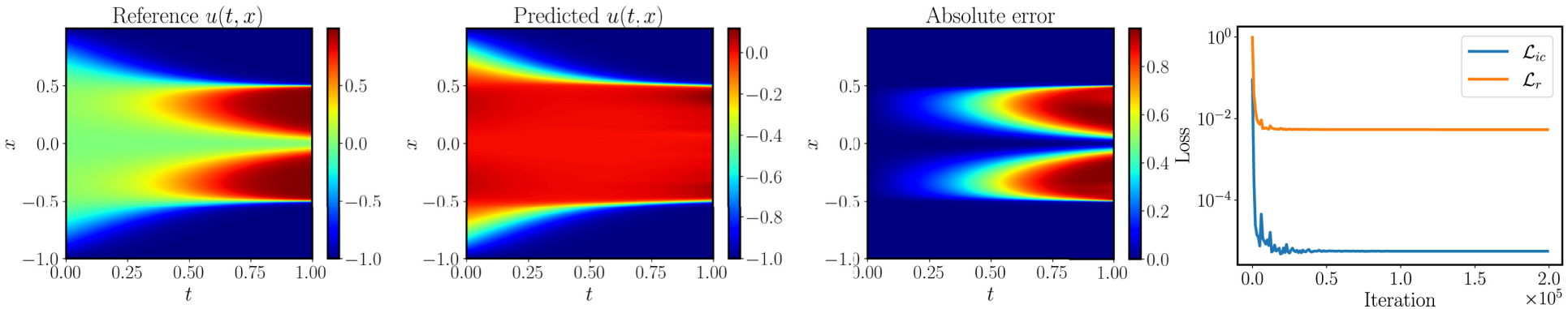




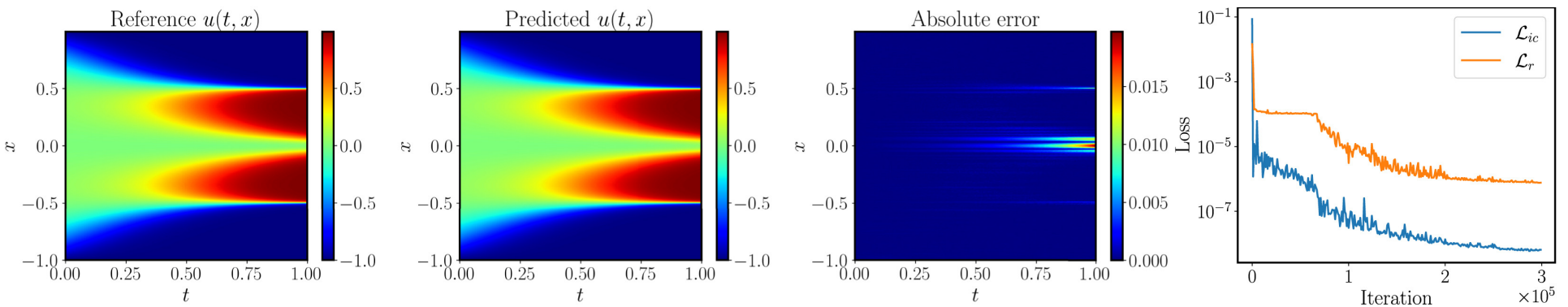
- A good-looking field may still give inaccurate derivatives, fluxes, or PDE residuals. Inspect those quantities directly when they matter physically.

Time-dependent training: respect the direction of information

Physical systems evolve *causally* from initial conditions forward in time. However, standard PINNs sample collocation points across the entire space-time domain and evaluate them simultaneously (cf. Wang et al. 2024, Allen-Cahn equation: $u_t - 0.0001u_{xx} + 5u^3 - 5u = 0$).



- *Time windows:* solve $[t_0, t_1]$, validate it, then transfer its final state to $[t_1, t_2]$. Interface errors propagate, so validate every transfer.
- *Causal weighting:* split the residual into time slabs $L_r^{(j)}$: late slabs receive weight only as earlier losses decrease.



High-order residuals: differentiate less when the model allows it

For a constant-stiffness beam with $EI\, u_{xxxx} = f$, with u the beam deflection, a strong PINN uses

$$r_\theta = EI\, u_{\theta,xxxx} - f, \qquad \nabla_\theta L_r = \frac{2}{N_r} \sum_i r_\theta(x_i)\, EI\, \nabla_\theta u_{\theta,xxxx}(x_i).$$

AD differentiates the implemented network accurately up to floating-point effects; it does not make high-order derivatives cheap, well-scaled, or physically accurate.

A mixed formulation reduces derivative order

Introduce curvature $q = u_{xx}$ and learn two outputs:

$$q_\theta - u_{\theta,xx} = 0, \qquad EI\, q_{\theta,xx} = f.$$

Only second spatial derivatives appear, but there is a new compatibility residual to balance. Translate the original boundary conditions consistently to the new variables.

A weak formulation moves derivatives to test functions

For clamped endpoints and admissible v with $v = v_x = 0$ at the endpoints, integration by parts twice gives

$$\int_0^1 EI\, u_{xx} v_{xx}\, dx = \int_0^1 fv\, dx \qquad \forall v.$$

Variational methods replace high-order point evaluations by test functions and quadrature: these introduce their own approximation and integration requirements.

A diagnosis should identify a mechanism before changing the method

Symptom	**Check**	**Response**
Boundary/initial error persists	Compare component gradient norms and directions	Scale/balance losses; enforce compatible data exactly
Low training residual, large local error	Fresh-point maps and targeted refinement	Resample; resolve layers; improve candidate coverage
Smooth field, incorrect derivatives	Inspect spectra, fluxes, derivative errors	Multiscale features; appropriate activations/formulation
Early-time error spoils later states	Evaluate each time slab and initial data	Causal weights or validated time windows
Slow loss decay or optimizer stall	Fixed-objective refinement; gradients and stopping reason	Better scaling; quasi-Newton/preconditioned methods
Noise strongly changes inverse estimates	Perturb data and inspect sensitivities	Regularization; better observations; report uncertainty

- Change one ingredient at a time in diagnostic experiments.
- Record both the new training behavior and its effect on independent physical errors, so that an improvement can be attributed to a mechanism.

Hard constraints (a short *tour*)

Hard constraints: restrict the family of admissible fields

- A boundary penalty asks training to make a constraint violation small:

$$\min_{\boldsymbol{\theta}} \; w_r L_r(\boldsymbol{\theta}) + w_b L_b(\boldsymbol{\theta}).$$

  For finite w_b, w_r, intermediate iterates and the final approximation can violate the boundary data. Increasing w_b, w_r also changes the optimization problem.
- A *hard constraint* is an identity built into the ansatz: every parameter vector produces a field that satisfies the chosen constraint. For Dirichlet data $u = g$ on Γ, use the conditioned network

$$\boxed{\tilde{u}_{\boldsymbol{\theta}}(\boldsymbol{x}) = a(\boldsymbol{x})\, u_{\boldsymbol{\theta}}(\boldsymbol{x}) + h(\boldsymbol{x})}, \qquad a|_\Gamma = 0, \quad h|_\Gamma = g.$$

  Thus $\tilde{u}_{\boldsymbol{\theta}}|_\Gamma = g$ *for every* $\boldsymbol{\theta}$.



What remains to be learned

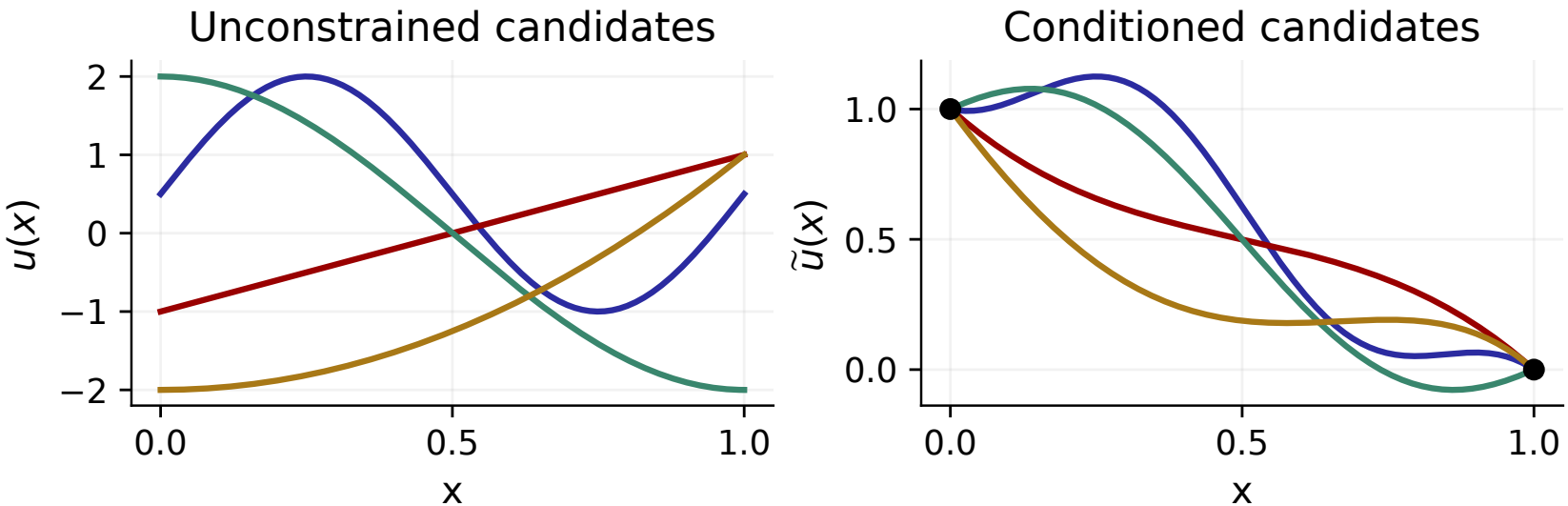
Evaluate the PDE using $\tilde{u}_{\boldsymbol{\theta}}$ and its derivatives. The corresponding boundary penalty is removed; the interior residual, other unenforced conditions, and any data loss remain. The construction enforces the selected constraint, not the entire PDE.

Exact Dirichlet values: what the boundary construction does

On $x \in [0,1]$, impose $u(0) = u_L$ and $u(1) = u_R$ with

$$h(x) = (1-x)u_L + xu_R, \qquad a(x) = x(1-x), \qquad \widetilde{u}_\theta = h + a\,u_\theta.$$

The network supplies an interior correction; the factor a makes that correction vanish at both endpoints. The curves below use $u_L = 1$, $u_R = 0$.



- For a Poisson residual, differentiate the *whole* conditioned field:

$$\tilde{u}_{\theta,xx} = h_{xx} + a\,u_{\theta,xx} + 2a_x u_{\theta,x} + a_{xx} u_\theta.$$

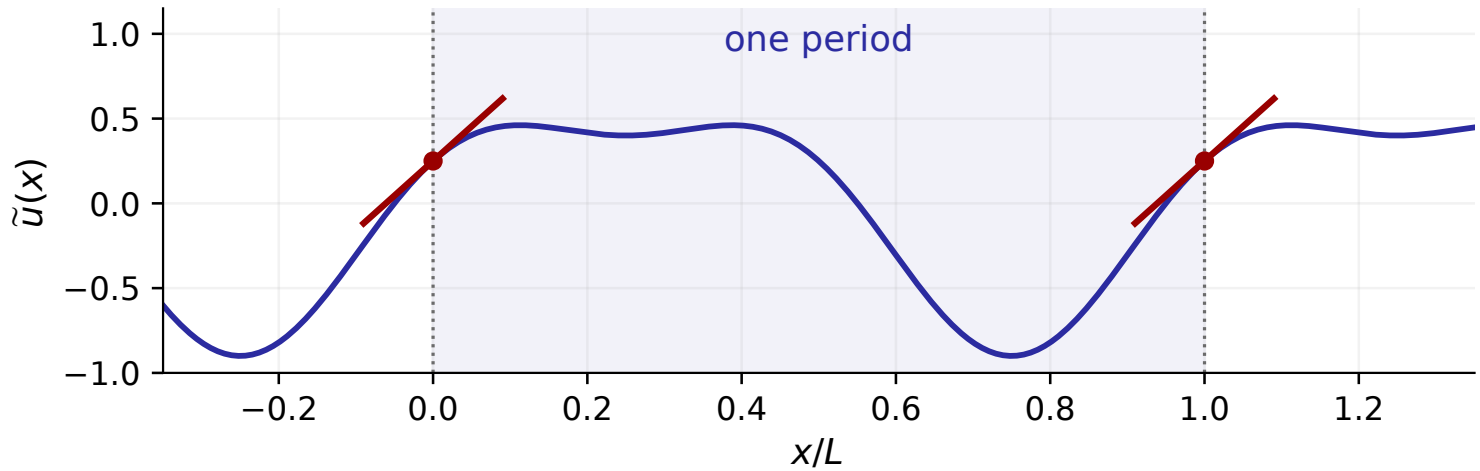
  The mask changes the differential operator seen during training.
- In several dimensions, h extends the boundary data and a vanishes on the prescribed boundary. Its smoothness must support the derivatives in F; a raw distance function may fail this requirement at corners or interior ridges.

Periodic inputs: identify coordinates before the network

To impose periodicity in x with period L, define

$$\boldsymbol{g}(x) = \left(\cos\frac{2\pi x}{L},\ \sin\frac{2\pi x}{L}\right), \qquad \boxed{\widetilde{u}_\theta(x,t) = u_\theta(\boldsymbol{g}(x),t)}.$$

Since $\boldsymbol{g}(x+L) = \boldsymbol{g}(x)$, the values at x and $x+L$ are identical for every $\boldsymbol{\theta}$. If the composition is k times differentiable, its derivatives through order k are periodic as well.



- The red tangents illustrate that the derivative agrees at the ends of a period; this construction supplies more than a penalty on endpoint values.
- A cosine-only input also enforces periodicity, but identifies x and $-x$ and therefore imposes an additional even symmetry. The sine–cosine pair retains phase information and can represent general periodic profiles.
- Additional integer harmonics can enrich the input while preserving period L. Choose their bandwidth with the spatial scales of the solution in mind.

Summary

- Neural networks are universal function approximators, and their potential as PDE solvers has long been recognized.
- However, their use became widespread only recently, under the name physics-informed neural networks (PINNs), prompted by advances in the deep learning revolution.
- In some cases, the PINN algorithm can produce inaccurate approximations due to the imbalance between the data-fitting and PDE-fitting terms in the loss function.
- Hard constraint methods can alleviate this issue by introducing physics knowledge into the architecture of the neural network itself, but this is not always possible if scalability is a must.
- A main issue is that standard deterministic neural networks do not provide a metric of confidence, i.e., *they do not know when they do not know*.
  A solution is to move towards probabilistic approaches, i.e., Bayesian neural networks and non-parametric models (e.g. Gaussian processes).

References

- F. Masi and I. Stefanou. Physics-informed and thermodynamics-based artificial neural networks. In Machine Learning in Geomechanics 2 (eds I. Stefanou and F. Darve). doi: 10.1002/9781394325665.ch3.
- U. Braga Neto, *ECEN 744: Scientific Machine Learning, Lecture 4: Physics-Informed Neural Networks*,`https://sciml.tamids.tamu.edu/ecen-744-scientific-machine-learning`.
- M. Raissi, P. Perdikaris and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* 378 (2019), 686–707. doi: doi:10.1016/j.jcp.2018.10.045.
- S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering* 421 (2024), p.116813. doi: 10.1016/j.cma.2024.116813.
- N.T. Andersen, and T. Matsubara. PINNs Failure Modes are Overfitting (2026). arXiv preprint arXiv:2605.30910.
- S. Khodakarami, V. Oommen, N.A. Daryakenari, M. Beekenkamp, G.E. Karniadakis. Spectral bias in physics-informed and operator learning: Analysis and mitigation guidelines. *Computer Methods in Applied Mechanics and Engineering* (2026), 461, p.119156. doi: 10.1016/j.cma.2026.119156